Multi-platform distributed systems with Aggregate Computing in Kotlin: the case of proximity messaging

Luca Marchi

ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA
Corso di Laurea Triennale in Ingegneria e Scienze Informatiche

Relatore: Prof. Pianini Danilo
Correlatore: Dott.ssa Cortecchia Angela

Sessione III Anno Accademico 2024-2025

Overview and Motivations

- **Context**: The world is increasingly populated by IoT and mobile devices that must communicate in dynamic environments.
- **Problem**: Centralized messaging architectures are fragile, difficult to scale, and unsuitable for pervasive and ubiquitous computing scenarios.
- **Motivation**: Resilient and more adaptive communication is needed.

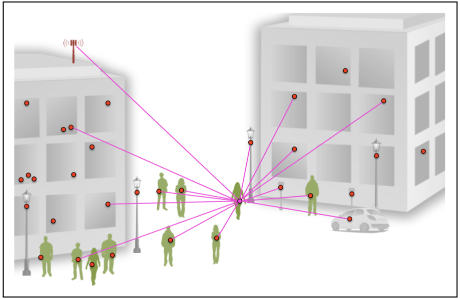

Figure: Example of a centralized system for proximity messaging.

Goals

This thesis project has two main goals:

Goal 1

Design and implement a decentralized, proximity-based messaging algorithm based on the principles of Aggregate Computing.

Goal 2

Develop a multiplatform (Android/iOS) mobile application that uses the algorithm as core logic.

Aggregate Computing: The paradigm

What is Aggregate Computing?

Aggregate Computing

Programming paradigm that treats large, interconnected systems of devices as a single, aggregate machine rather than focusing on individual components.

Key concepts

- **Collective behavior**: The focus is on programming the group, or "aggregate," as a whole.
- **Abstraction**: It provides a higher level of abstraction that hides the complexity of managing individual devices in a vast and dynamic network.
- **Logic**: The program logic is expressed as the manipulation of data structures that extend spatially and temporally (**Field Calculus**).

Project Specifications

Architecture

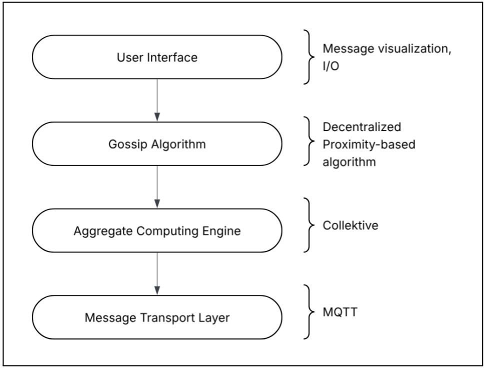

Figure: High-level overview of the architecture.

Technologies

- **AC framework**: Collektive
- **Programming language**: Kotlin
- **Multi-platform framework**: KMP
- **GUI**: Compose Multiplatform
- **Simulator**: Alchemist Simulator
- **Transport Protocol**: MQTT

Algorithm (Gossip-Gradient)

Validation & CI

Mobile Application (Echo)

Conclusions & Future Development

The End